RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Course Code	AD23532	**Course Name**	Principles of Data Science
Course Type	Lab Oriented Theory	**Course Category**	Professional Core (PC)
Regulation	2023	**Academic Year**	2026-27 (ODD)
Degree and Branch	B.Tech. Artificial Intelligence and Data Science	**Batch**	2024-2028
Submission Date	03-10-2026	**Semester**	V

Review - 3

Model Building and Comparison Analysis

Food Waste Prediction and Meal-Planning Optimization for Event Catering

Item	Detail
Student	PRAVEEN RAMANAN K
Register number	241801214
Deliverables	Technical report (this document); notebooks/Review3_Model_Building_and_Comparison.ipynb; src/ (pipeline, extended EDA, training); models/ (trained artefacts); app/streamlit_app.py (dashboard)

1. Introduction and Scope of Review 3

Review 1 defined the problem: forecast food waste for an event-catering booking before the event, grade the booking's waste risk, and turn that grade into a preparation action and, where warranted, a donation-partner alert. Review 2 audited, cleaned and explored the Kaggle dataset Food Wastage Data in Restaurant, built a leakage-safe train/test partition and engineered pre-event features.

Review 3 completes the project. It builds, tunes and compares machine-learning models for both learning tasks, selects the final models on quantitative evidence, and delivers a working decision-support dashboard. In line with the review brief, it covers:

- **Model selection and justification** -- seven algorithm families per task (Section 4).
- **Model development** -- leakage-safe 80/20 split plus 5-fold cross-validation (Section 5).
- **Model evaluation** -- MAE, MSE, RMSE and R2 for regression; accuracy, precision, recall, F1, ROC-AUC and confusion matrices for classification (Section 6).
- **Hyperparameter tuning** -- Grid Search and Randomized Search with before/after comparison (Section 7).
- **Performance comparison** -- tables, charts, strengths and limitations (Section 8).
- **Final model selection** -- a selection rule fixed in advance, plus a corrected significance test (Section 9).
- **End-to-end product** -- the Streamlit dashboard driven by the trained models (Section 10).

Before modelling, the exploratory analysis is extended from the two dominant drivers studied in Review 2 to every column of the dataset (Section 3), because those findings shape the feature set and the model families.

1.1 Learning tasks and data recap

Table 1 -- Data and task summary

Item	Detail
Raw data	1,782 records x 11 columns (Kaggle, Food Wastage Data in Restaurant)
Cleaned data	1,618 unique bookings after removing exact duplicates
Partition	1,294 training / 324 test bookings (80/20, seed 42), split before any target-derived statistic
Regression target	wastage_food_amount (units of food wasted per booking)
Classification target	Risk tier: LOW < 20 <= MEDIUM < 35 <= HIGH (training-set quartiles)
Tier distribution (train)	MEDIUM 50.9%, HIGH 34.2%, LOW 15.0%
Design matrix	33 columns: 4 numeric, 3 pricing/service encodings, one-hot encodings of 7 categorical columns and a 4-band guest-scale feature
Noise ceiling	Implied R2 ceiling ~ 0.9328 from identical-input records with differing targets (Review 2, Section 6.4)

2. Feature Set Carried into Modelling

The Review 2 feature set uses only information known at booking time: number_of_guests, quantity_of_food, food_per_guest, surplus_units, pricing_ordinal, is_high_price, is_sit_down and guest_scale, together with the categorical columns. Expected Waste per Guest is derived from the target

and stays excluded. Two automated leakage guards remain in place: no feature name may contain a target-derived substring, and no numeric feature may correlate with the target above |r| = 0.95.

One change was made for Review 3 on the basis of the extended EDA. Preparation method is now one-hot encoded in full, not represented only by the sit-down flag. Finger Food and Buffet bookings differ systematically (Section 3), and the change lifts the 5-fold CV R2 of XGBoost by about 0.008. The design matrix grows from 30 to 33 columns.

3. Extended Exploratory Data Analysis

Review 2 concentrated on pricing tier and preparation method. This section examines every column: its effect on the waste amount, on the waste rate (waste as a share of food prepared), and on the risk tier, and how the columns relate to each other. All statistics use the training partition only.

3.1 Waste across every categorical level

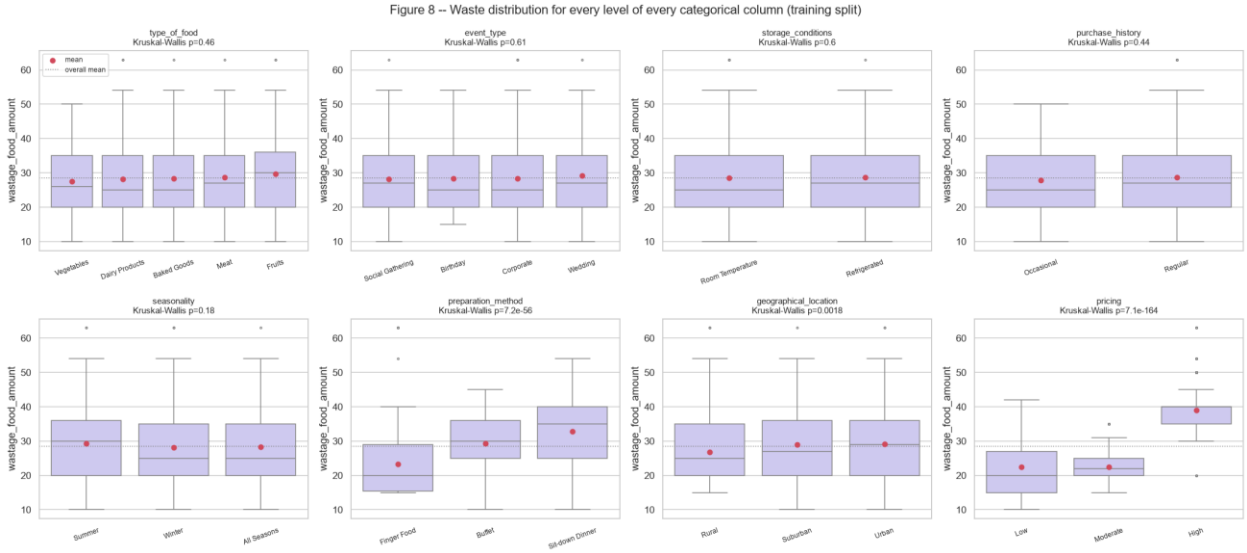


Figure R1 -- Waste distribution for every level of the eight categorical columns, with Kruskal-Wallis p-values.

Only two columns move the distribution visibly. High-price bookings average 39.0 units, against 22.5 (Moderate) and 22.5 (Low). Sit-down Dinner averages 32.9, against 29.2 for Buffet and 23.3 for Finger Food. For the other six columns, the medians and interquartile ranges overlap almost completely.

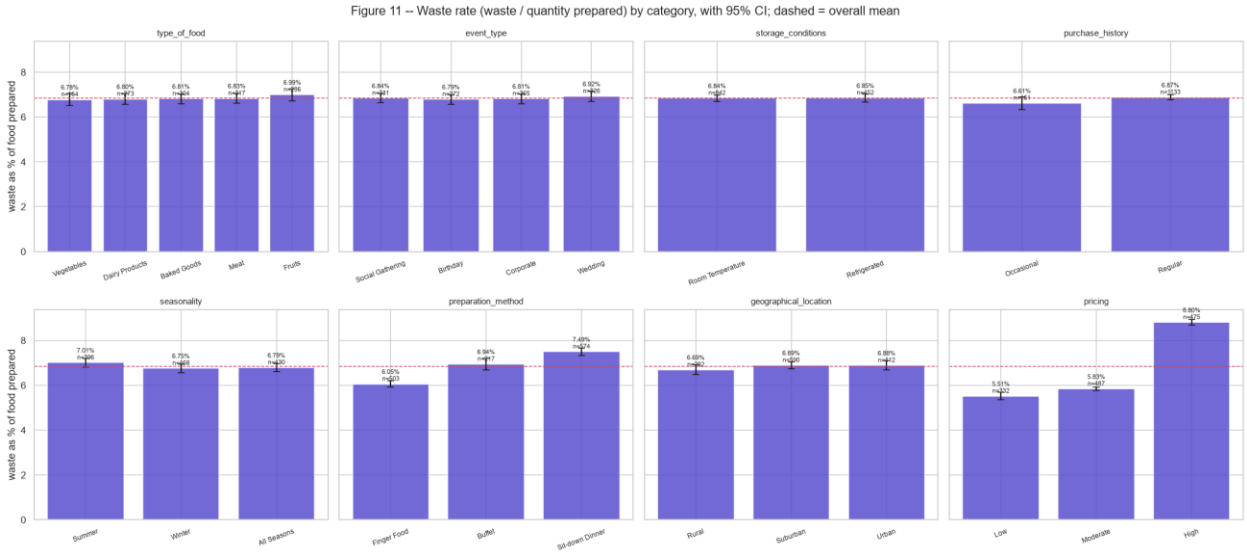


Figure R2 -- Waste rate (waste / quantity prepared) per level with 95% confidence intervals.

Normalising by the quantity prepared does not change the picture: High-price bookings waste 8.80% of the food prepared, against 5.51% for Low-price bookings. The pricing effect is therefore not just a consequence of premium events preparing more food.

3.2 Association with the risk tier

Table 2 -- Chi-square test and bias-corrected Cramer's V between each column and the risk tier

feature	chi2	dof	p_value	cramers_v	association
pricing	**783.55**	**4**	**2.81e-168**	**0.549**	**Strong**
guest_scale	**386.84**	**6**	**1.89e-80**	**0.384**	**Strong**
preparation_method	**267.02**	**4**	**1.40e-56**	**0.319**	**Strong**
geographical_location	15.8	4	3.29e-03	0.068	Weak/none
event_type	17.03	6	9.18e-03	0.065	Weak/none
storage_conditions	6.38	2	4.12e-02	0.058	Weak/none
seasonality	11.16	4	2.48e-02	0.053	Weak/none
purchase_history	2.91	2	2.33e-01	0.027	Weak/none
type_of_food	4.09	8	8.49e-01	0	Weak/none

Figure 13 -- Risk-tier composition within each categorical level

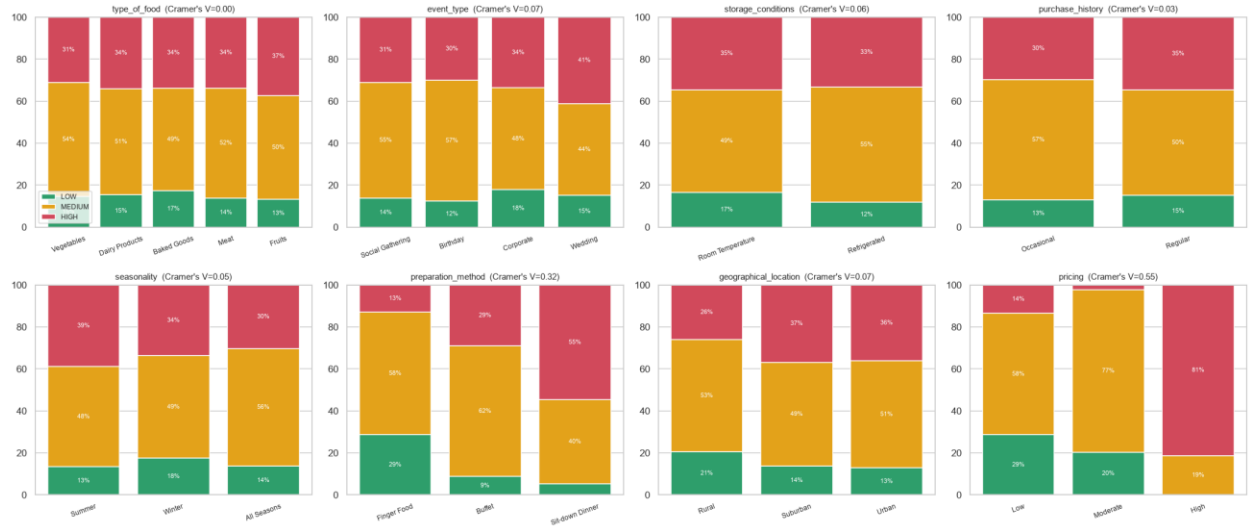


Figure R3 -- LOW / MEDIUM / HIGH composition within every categorical level.

Three columns are strongly associated with the risk tier: pricing, the guest-scale band and preparation method. Location reaches statistical significance, but its effect is small (V < 0.1). Rural bookings have fewer HIGH-risk tiers, partly because they prepare less food on average. Food type, purchase history, seasonality, storage and event type have no practically meaningful association.

3.3 Interactions between columns

Table 3 -- Two-way ANOVA with sequential sums of squares (factor a, factor b given a, interaction)

factor_a	factor_b	F_a	p_a	F_b	p_b	F_ab	p_ab	partial_eta2_ab
event_type	type_of_food	0.51	6.78e-01	1.12	3.45e-01	0.4	9.64e-01	0.0038
seasonality	storage_conditions	1.62	1.98e-01	0.01	9.34e-01	0.07	9.34e-01	0.0001

geographical_location	pricing	14.25	7.54e-07	949.47	6.53e-254	1.86	1.15e-01	0.0058
purchase_history	event_type	0.81	3.67e-01	0.65	5.81e-01	0.13	9.42e-01	0.0003
pricing	**preparation_method**	**1,106**	**4.65e-280**	**118.34**	**6.72e-48**	**11.54**	**3.28e-09**	**0.0347**

Figure 12 -- Mean waste for pairs of secondary categorical columns

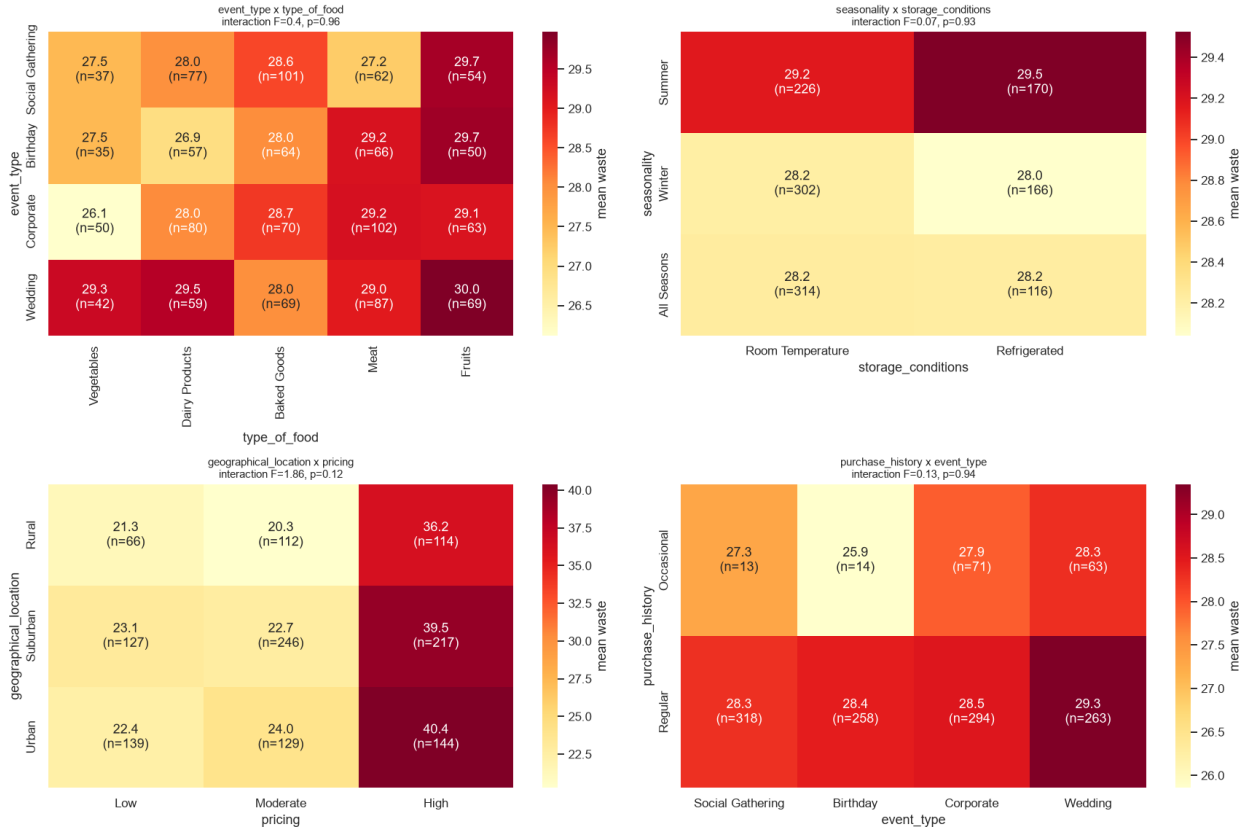


Figure R4 -- Mean waste for four pairs of secondary columns (cell counts in brackets).

Pricing x preparation method is the only pair with a significant interaction. It reproduces the Review 2 result, F(4, 1285) = 11.54, and corresponds to a rank reversal of the service styles across price tiers. Event type x food type, season x storage and purchase history x event type have no detectable main or interaction effects. The one detectable non-pricing effect is location, and it is a main effect only: the location x pricing interaction is not significant.

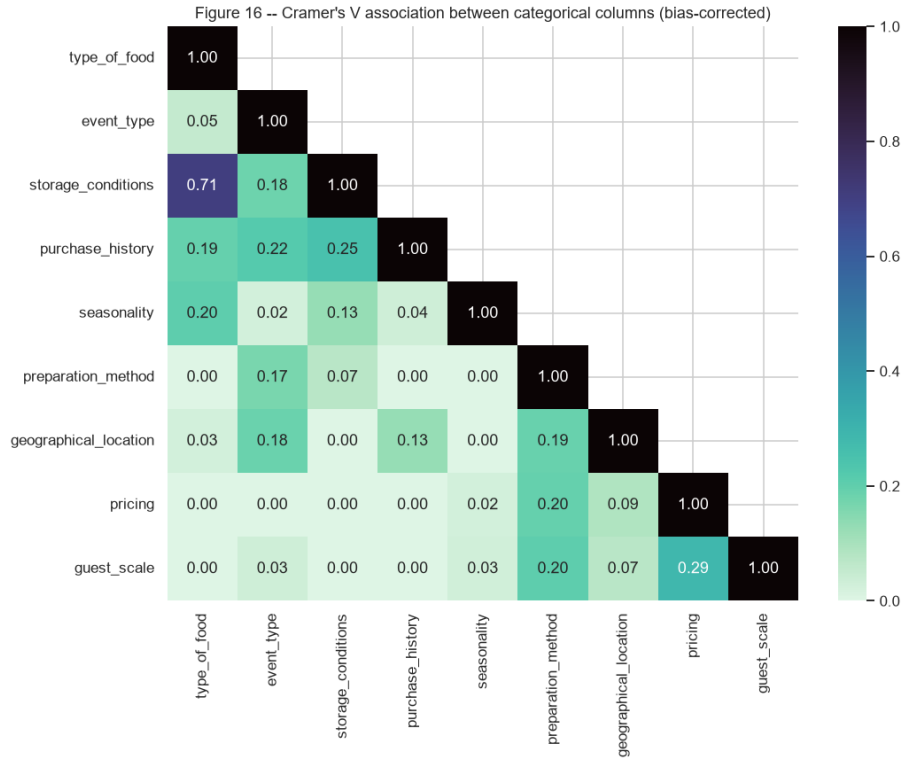


Figure R5 -- Cramer's V between all categorical columns (bias-corrected).

Some secondary columns are clearly related to each other. Food type and storage condition are strongly associated (V = 0.71), and storage, purchase history, event type and seasonality show moderate links (V ~ 0.2). Pricing is related to the guest-scale band (V = 0.29) and to preparation method (V = 0.20). None of the inter-related secondary columns is associated with waste or with the risk tier (Table 2), so their redundancy hides no signal. Tree ensembles are unaffected by such collinearity.

3.4 Numeric structure

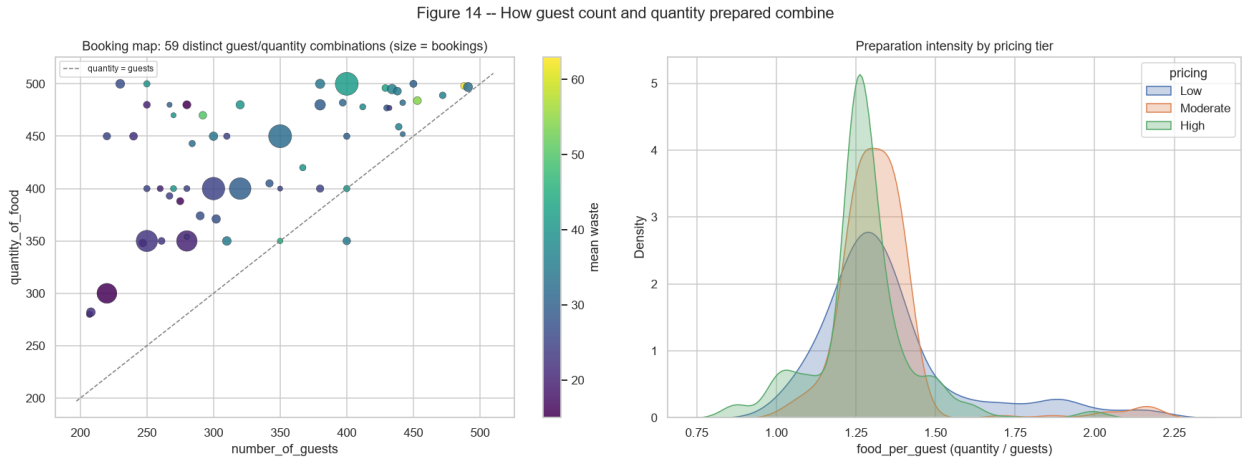


Figure R6 -- Booking map of guests x quantity coloured by mean waste (left); preparation intensity by pricing tier (right).

Figure 19 -- Numeric and engineered features coloured by risk tier

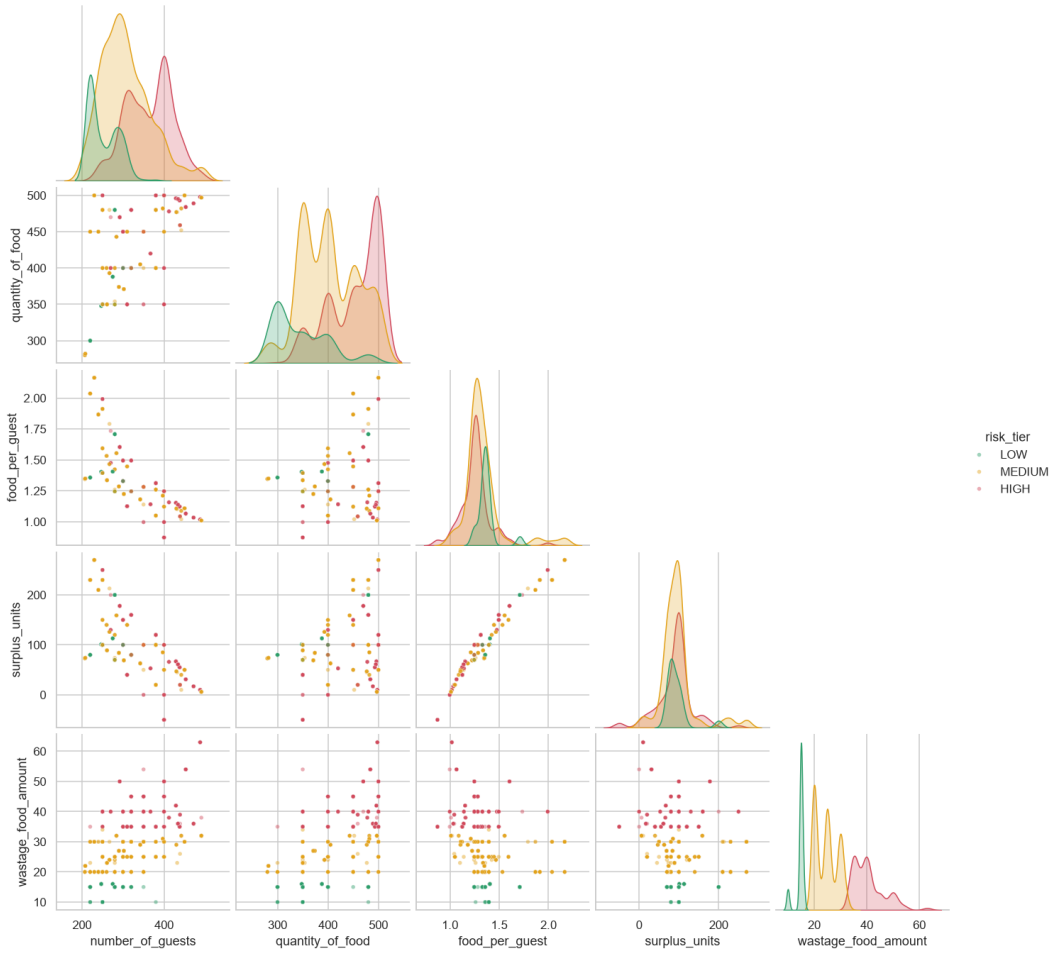


Figure R7 -- Numeric and engineered features coloured by risk tier.

The bookings sit on a small grid of guest/quantity combinations. The risk tiers separate along guest count and quantity, but they overlap heavily unless pricing is also known. This is why no single numeric feature is sufficient and why the models must combine the numeric features with pricing.

Figure 15 -- Event-type profile

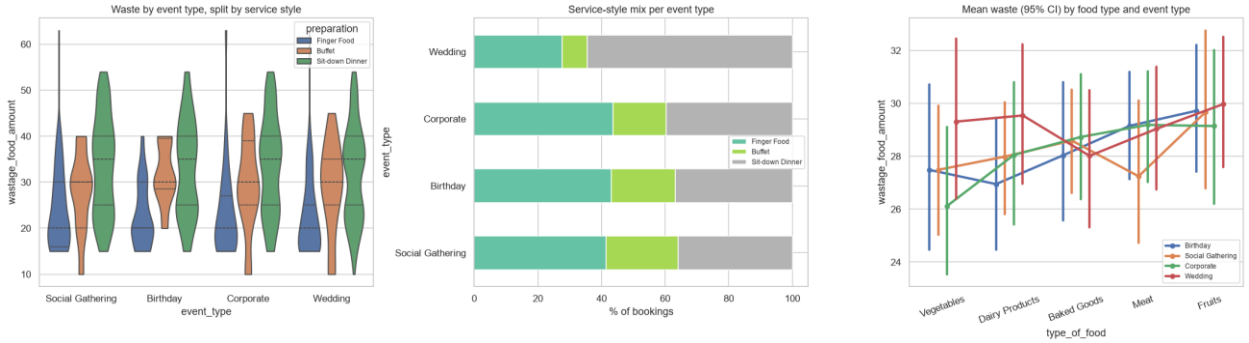


Figure R8 -- Event-type profile: waste by service style, service mix, and food type.

Figure 17 -- Cumulative distribution of waste; dotted lines = risk-tier cut points

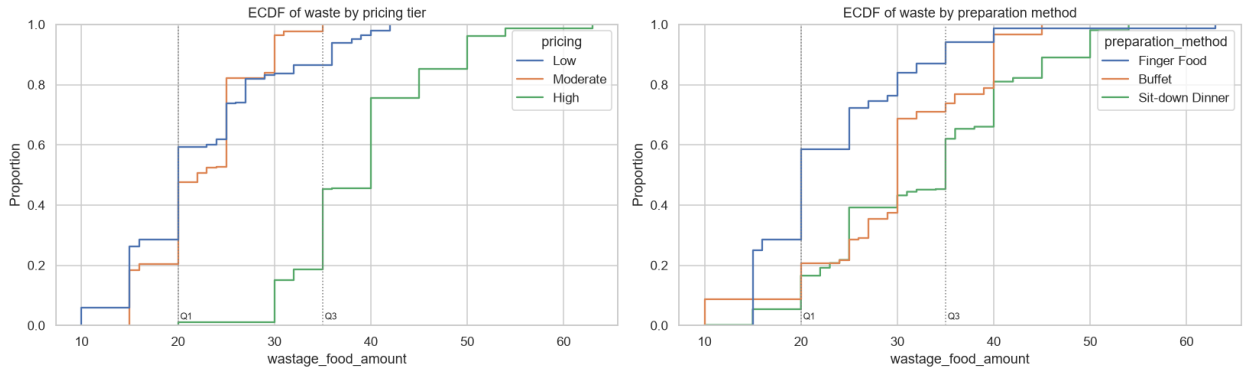


Figure R9 -- Cumulative distribution of waste by pricing tier and service style, with the tier cut points.

Figure 18 -- Waste per guest (descriptive only; target-derived, never a model input)

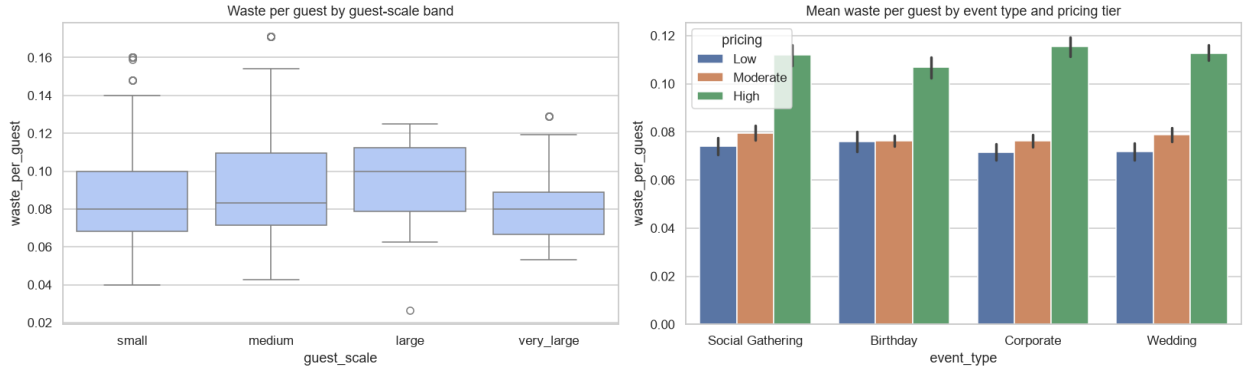


Figure R10 -- Waste per guest by guest-scale band and by event type x pricing (descriptive only).

Figure 20 -- Operational context columns: season, location, storage

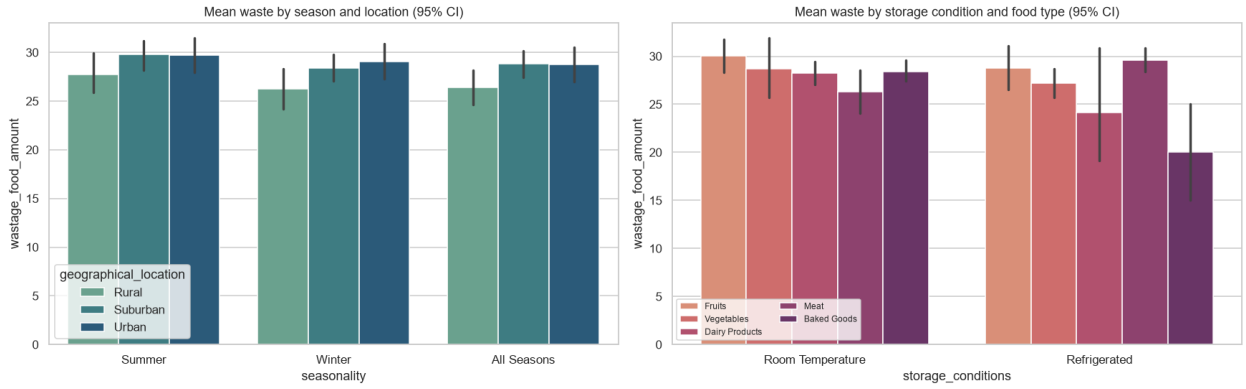


Figure R11 -- Operational-context columns: season x location and storage x food type.

3.5 Implications for modelling

- Signal is concentrated in pricing, preparation method, guests and quantity, with a non-additive pricing x service interaction. Tree ensembles, which learn interactions automatically, should outperform additive linear models.
- Weak columns are kept rather than dropped. Regularised ensembles can ignore them at little cost, and the final permutation-importance analysis (Section 9) checks this explicitly.
- The risk tier is imbalanced (LOW ~15%) and many targets sit exactly on the cut points, so macro-F1 rather than accuracy is the primary classification metric, and class weighting is part of the tuning search.
- The target takes few distinct values, which favours histogram-based boosting (XGBoost with tree_method='hist', HistGradientBoosting).

4. Model Selection and Justification

Seven algorithms are compared for each task. Together they span baseline, linear, instance-based, kernel, bagging and boosting families, so the comparison says something about the shape of the problem and not only about which library performs best.

Table 4 -- Candidate models and the reason each was chosen

task	model	rationale
regression	Naive (mean)	Floor: any useful model must beat predicting the training mean.
regression	Linear Regression	Interpretable additive baseline; tests how far a linear fit gets (tuned as Ridge).
regression	KNN	Non-parametric; bookings with similar guests/quantity/tier should waste alike.
regression	SVR	RBF kernel captures smooth non-linearity; epsilon-insensitive loss resists label noise.
regression	Random Forest	Bagged trees model the pricing x service interaction without manual terms.
regression	HistGradientBoosting	Histogram-binned gradient boosting; fast, strong on tabular data with few distinct values.
regression	XGBoost	Regularised gradient boosting with histogram splits; usually the strongest tabular learner.
classification	Dummy (most frequent)	Floor: always predicts the majority tier (MEDIUM).
classification	Logistic Regression	Linear, calibrated probabilities; interpretable multinomial baseline.
classification	KNN	Instance-based; similar bookings should share a tier.
classification	SVM (RBF)	Maximum-margin non-linear boundaries between adjacent tiers.
classification	Random Forest	Robust ensemble; class weighting can lift the minority LOW tier.
classification	HistGradientBoosting	Histogram-binned gradient boosting with native multi-class loss.
classification	XGBoost	Regularised multi-class boosting (softprob) with histogram splits.

XGBoost is configured with the histogram tree method (tree_method = 'hist'), which bins each feature into at most max_bin buckets before searching for splits. scikit-learn's HistGradientBoosting applies the same idea in an independent implementation. Comparing the two shows whether XGBoost's advantage comes from its regularisation terms or from histogram boosting itself.

5. Model Development

Table 5 -- Training and validation protocol

Item	Detail
Hold-out split	80/20 random split (seed 42), made before risk tiers or segment benchmarks were computed; test set used once, for final scoring only
Cross-validation	5-fold KFold (regression) and StratifiedKFold (classification), shuffled with seed 42, on the training partition; identical folds for every model
Scaling	StandardScaler applied inside a Pipeline for Linear/Logistic, KNN and SVM, so each CV fold fits its own scaler; tree models use the unscaled matrix
Labels	LOW / MEDIUM / HIGH encoded as 0 / 1 / 2 for every classifier, so XGBoost and scikit-learn share one label space
Two stages	Baseline (library defaults), then tuned (Grid or Randomized search on the

	same 5 folds)
Selection rule	Highest tuned 5-fold CV score (R2 / macro-F1), fixed before inspecting test results
Significance	Nadeau-Bengio corrected resampled t-test, best vs runner-up, on 5x3 repeated CV
Reproducibility	python src/pipeline.py; python src/train_models.py; python src/build_notebooks.py

6. Model Evaluation

Regression is evaluated with MAE, MSE, RMSE and R2 on the test set, together with cross-validated R2 (mean +/- standard deviation over the 5 folds). Classification is evaluated with accuracy, balanced accuracy, macro precision, macro recall, macro and weighted F1, one-vs-rest ROC-AUC, log-loss and the recall of the HIGH tier. HIGH recall matters most operationally, because a missed HIGH booking means a missed donation alert. Accuracy is reported but not used for selection: always predicting MEDIUM already scores 50% on the test set.

6.1 Baseline models (default hyperparameters)

Table 6 -- Regression baseline metrics

model	cv_R2_mean	cv_R2_std	test_MAE	test_MSE	test_RMSE	test_R2
Random Forest	0.903	0.0167	1.7845	8.7271	2.9542	0.919
HistGradientBoosting	0.9026	0.0147	2.0214	8.4849	2.9129	0.9213
XGBoost	0.8934	0.0156	2.1442	9.6622	3.1084	0.9103
KNN	0.7864	0.0168	3.5648	23.1814	4.8147	0.7849
Linear Regression	0.767	0.0238	4.0272	28.5186	5.3403	0.7353
SVR	0.7479	0.0277	3.8184	30.4013	5.5137	0.7179
Naive (mean)	-0.0058	0.0048	8.6218	107.955	10.3901	-0.0019

Table 7 -- Classification baseline metrics

model	cv_macro_F1_mean	cv_macro_F1_std	test_accuracy	test_precision_macro	test_recall_macro	test_macro_F1	test_ROC_AUC_ovr
Random Forest	0.8887	0.0231	0.9012	0.8891	0.846	0.8639	0.9672
XGBoost	0.8791	0.0213	0.9043	0.8869	0.8599	0.8719	0.9713
HistGradientBoosting	0.8782	0.0174	0.8858	0.8559	0.8302	0.8415	0.9724
SVM (RBF)	0.8445	0.0291	0.8519	0.8483	0.803	0.8214	0.938
KNN	0.8266	0.0106	0.8364	0.815	0.7841	0.7974	0.9185
Logistic Regression	0.7847	0.0237	0.7994	0.809	0.7491	0.7703	0.9207
Dummy (most frequent)	0.2247	0.0005	0.5	0.1667	0.3333	0.2222	0.5

Even untuned, the three tree ensembles are well ahead of the other families. Random Forest is the strongest default model for both tasks, while XGBoost with default settings (depth 6, learning rate 0.3) over-fits slightly and trails it.

7. Hyperparameter Tuning and Model Optimisation

Grid Search is used where the space is small and discrete (Ridge alpha, KNN neighbours/weights/metric, SVM C, gamma, epsilon and class weight). Randomized Search with continuous distributions is used for the tree ensembles, where a full grid would be infeasible: 25 configurations for Random Forest, 40 for HistGradientBoosting and 50 for XGBoost, each scored over the same 5 folds.

Table 8 -- Search method and best hyperparameters found

task	model	method	configs	time (s)	best hyperparameters
regression	Linear Regression	grid	8	0.3	alpha=1
regression	KNN	grid	28	0.8	n_neighbors=3, p=1, weights=uniform
regression	SVR	grid	48	4	C=30, epsilon=0.5, gamma=scale
regression	Random Forest	random	25	20.8	n_estimators=700, min_samples_leaf=2, max_features=0.5, max_depth=10
regression	HistGradientBoosting	random	40	23.5	l2_regularization=5.141, learning_rate=0.06793, max_depth=3, max_iter=445, max_leaf_nodes=7, min_samples_leaf=6
regression	XGBoost	random	50	10	colsample_bytree=0.8345, gamma=1.728, learning_rate=0.02188, max_bin=256, max_depth=5, min_child_weight=3, n_estimators=341, reg_alpha=0.2441, reg_lambda=0.2556, subsample=0.9762
classification	Logistic Regression	grid	16	0.9	C=30, class_weight=None
classification	KNN	grid	28	0.7	n_neighbors=5, p=1, weights=uniform
classification	SVM (RBF)	grid	48	9.8	C=10, class_weight=None, gamma=scale
classification	Random Forest	random	25	39.8	n_estimators=700, min_samples_leaf=2, max_features=0.5, max_depth=16, class_weight=None
classification	HistGradientBoosting	random	40	73.7	class_weight=balanced, l2_regularization=2.903, learning_rate=0.06261, max_depth=3, max_iter=447, max_leaf_nodes=7, min_samples_leaf=43
classification	XGBoost	random	50	25.1	colsample_bytree=0.663, gamma=0.4405, learning_rate=0.1123, max_bin=256, max_depth=4, min_child_weight=4, n_estimators=860, reg_alpha=0.004415, reg_lambda=3.889, subsample=0.7547

7.1 Before vs after optimisation

Table 9 -- Regression: CV and test R2 before and after tuning

model	cv_R2_baseline	cv_R2_tuned	test_R2_baseline	test_R2_tuned	cv_R2_gain	test_R2_gain
HistGradientBoosting	0.9026	0.9091	0.9213	0.9279	0.0065	0.0066
KNN	0.7864	0.7927	0.7849	0.7956	0.0063	0.0107
Linear Regression	0.767	0.7671	0.7353	0.7357	0.0001	0.0004
Naive (mean)	-0.0058	-0.0058	-0.0019	-0.0019	0	0
Random Forest	0.903	0.9121	0.919	0.9244	0.0091	0.0054
SVR	0.7479	0.8256	0.7179	0.8154	0.0777	0.0975
XGBoost	0.8934	0.9181	0.9103	0.9316	0.0247	0.0213

Table 10 -- Classification: CV and test macro-F1 before and after tuning

model	cv_macro_F1_baseline	cv_macro_F1_tuned	test_macro_F1_baseline	test_macro_F1_tuned	cv_macro_F1_gain	test_macro_F1_gain
Dummy (most frequent)	0.2247	0.2247	0.2222	0.2222	0	0
HistGradientBoosting	0.8782	0.8857	0.8415	0.8634	0.0075	0.0219
KNN	0.8266	0.8357	0.7974	0.798	0.0091	0.0006
Logistic Regression	0.7847	0.8015	0.7703	0.7539	0.0168	-0.0164
Random Forest	0.8887	0.8908	0.8639	0.8786	0.0021	0.0147
SVM (RBF)	0.8445	0.8627	0.8214	0.8384	0.0182	0.017
XGBoost	0.8791	0.8863	0.8719	0.8893	0.0072	0.0174

Figure 27 -- Effect of hyperparameter tuning (before -> after)

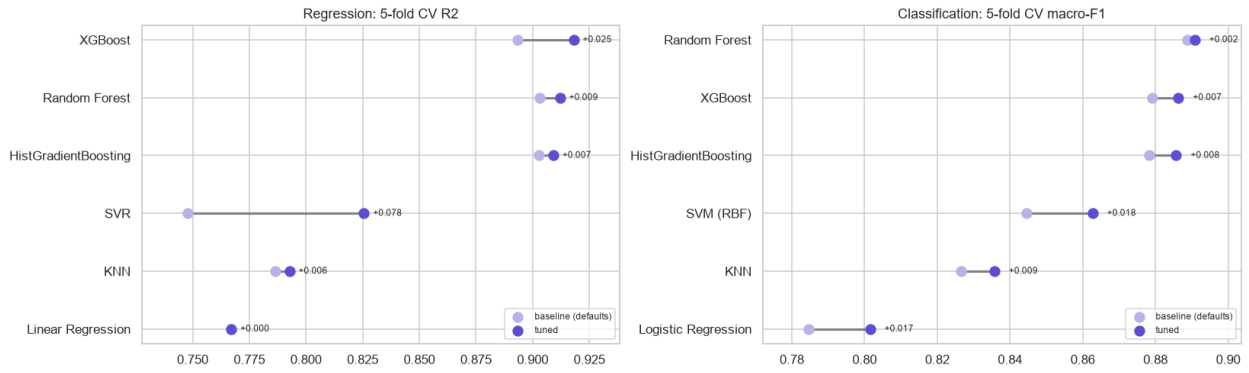


Figure R12 -- Effect of tuning on cross-validated scores (baseline -> tuned).

Tuning has the largest effect where the defaults were mismatched to the data. SVR gains +0.078 CV R2, because the default C = 1 is far too small for a target measured in units. XGBoost gains +0.025 CV R2 and +0.021 test R2. The search settles on max_depth = 5, learning_rate = 0.022, n_estimators = 341 and gamma = 1.73: shallower trees, a much smaller step size, and a minimum split gain that prunes splits fitted to label noise. These gains move XGBoost from third place to first in regression. Random Forest and HistGradientBoosting improve only marginally, because their defaults were already close to optimal.

Figure 30 -- XGBoost (hist) hyperparameter search: every sampled configuration

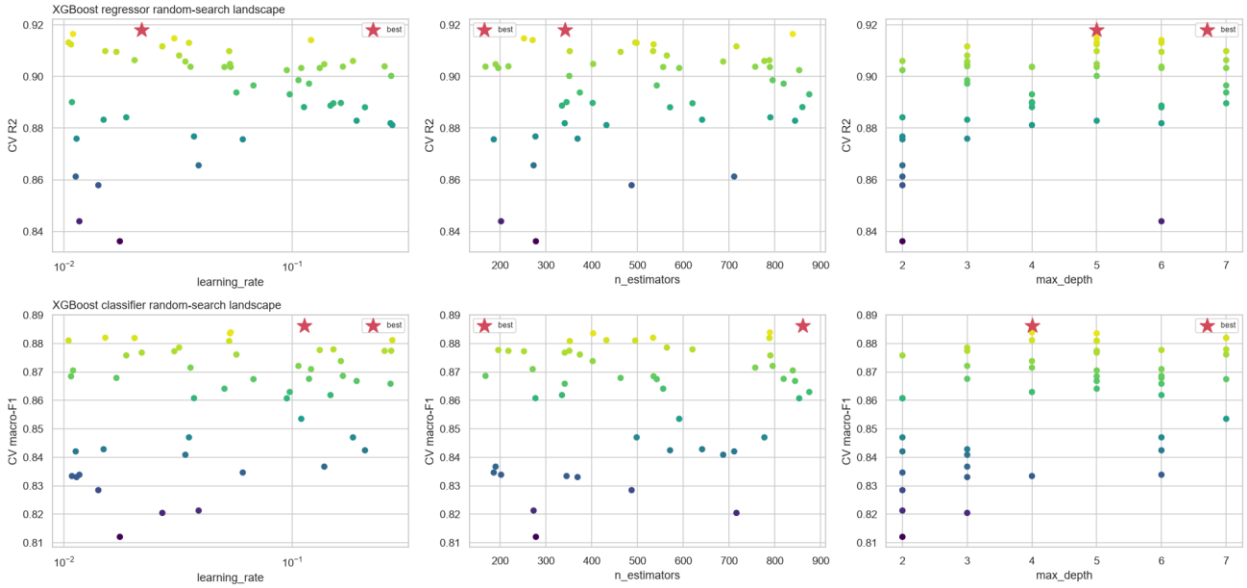


Figure R13 -- Every configuration sampled by the XGBoost random search (star = selected).

8. Model Performance Comparison

8.1 Regression

Table 11 -- Tuned regression models (sorted by CV R2; final model highlighted)

model	cv_R2_mean	cv_R2_std	test_MAE	test_MSE	test_RMSE	test_R2	test_within_5_units_pct	fit_time_s
XGBoost	**0.9181**	**0.0122**	**1.887**	**7.3669**	**2.7142**	**0.9316**	**89.8148**	**0.2575**
Random Forest	0.9121	0.0147	1.8914	8.1405	2.8532	0.9244	89.8148	1.8311
HistGradientBoosting	0.9091	0.0133	2.0157	7.7655	2.7867	0.9279	91.0494	0.8101
SVR	0.8256	0.0216	2.9948	19.8877	4.4596	0.8154	82.4074	0.1401
KNN	0.7927	0.023	3.392	22.0202	4.6926	0.7956	80.5556	0.0091
Linear Regression	0.7671	0.0237	4.0295	28.4835	5.337	0.7357	70.9877	0.011
Naive (mean)	-0.0058	0.0048	8.6218	107.955	10.3901	-0.0019	29.6296	0.0007

Figure 21 -- Regression model comparison, baseline vs tuned

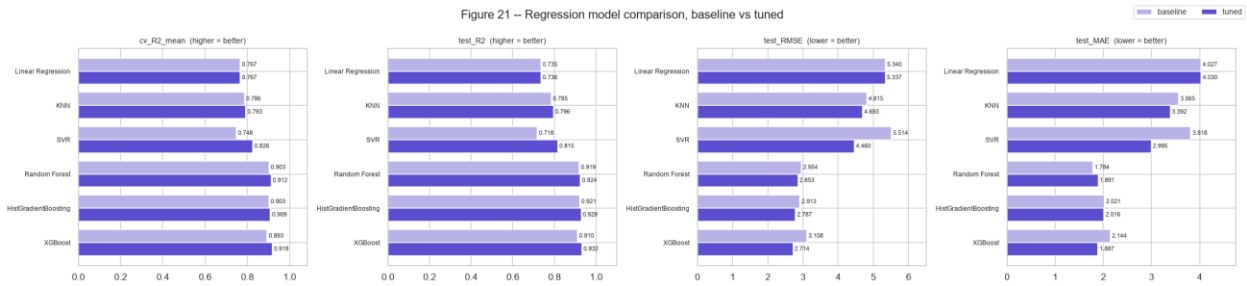


Figure R14 -- Regression metrics, baseline vs tuned.

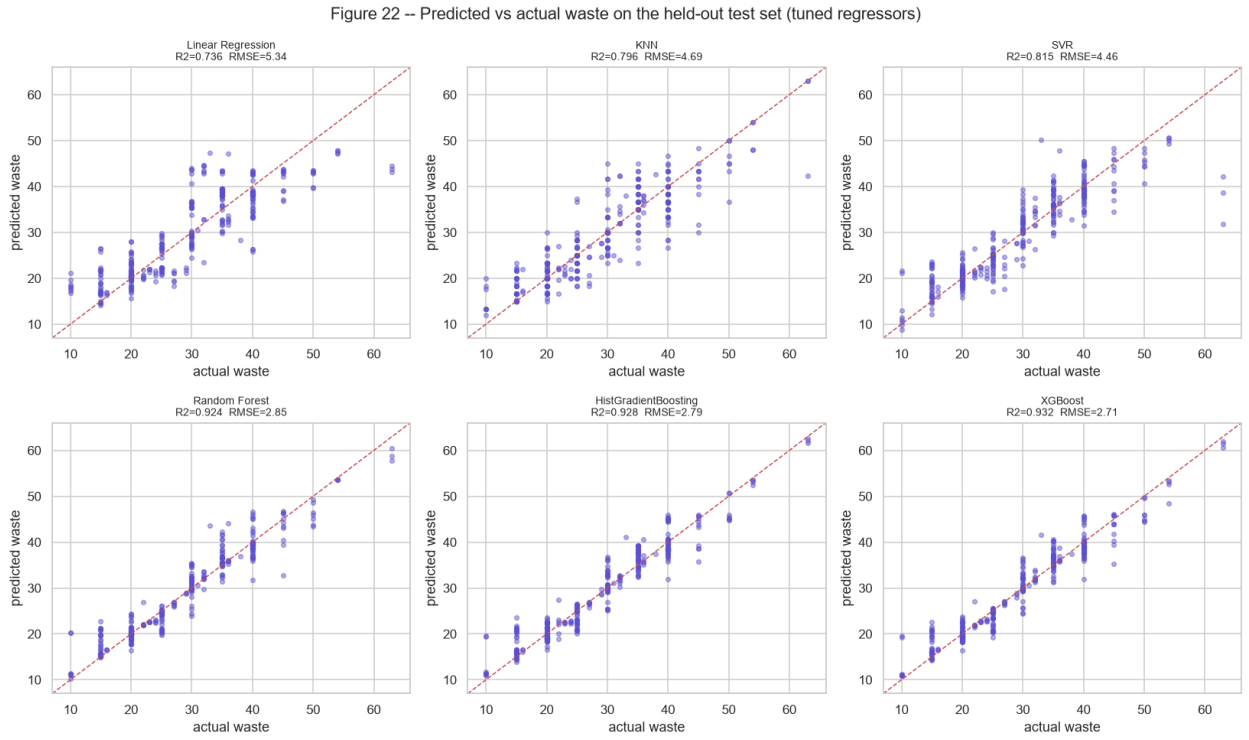


Figure R15 -- Predicted vs actual waste on the test set for each tuned regressor.

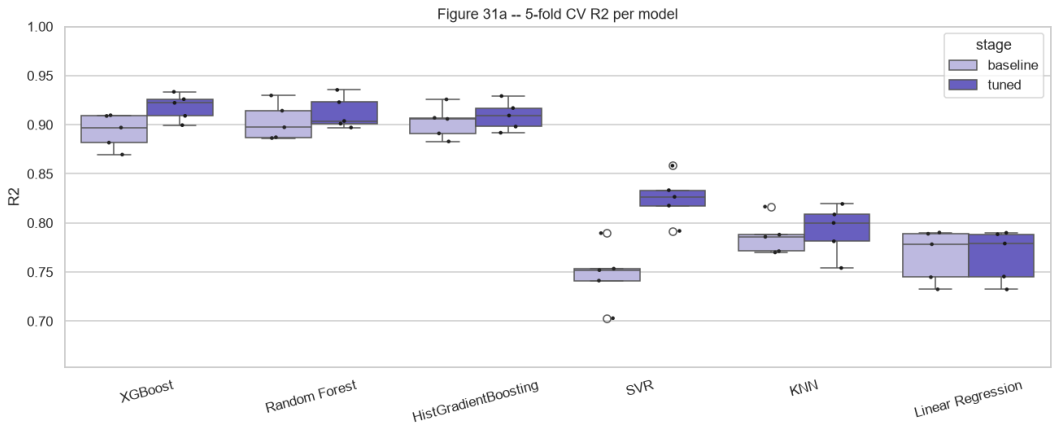


Figure R16 -- Fold-level CV R2: spread as well as level.

8.2 Classification

Table 12 -- Tuned classification models (sorted by CV macro-F1; final model highlighted)

model	cv_F1	cv_F1_std	accuracy	balanced_accuracy	precision_macro	recall_macro	macro_F1	ROC_AUC_ovr	HIGH_recall
Random Forest	**0.8908**	**0.0256**	**0.9105**	**0.8585**	**0.908**	**0.8585**	**0.8786**	**0.9746**	**0.9492**
XGBoost	0.8863	0.0201	0.9105	0.8727	0.9108	0.8727	0.8893	0.9745	0.9237
HistGradientBoosting	0.8857	0.0206	0.8858	0.8854	0.8485	0.8854	0.8634	0.9721	0.9407
SVM (RBF)	0.8627	0.0249	0.8765	0.8162	0.8738	0.8162	0.8384	0.9425	0.9153
KNN	0.8357	0.0138	0.8364	0.7904	0.8072	0.7904	0.798	0.927	0.8644
Logistic Regression	0.801	0.028	0.784	0.7436	0.7693	0.7436	0.7539	0.9263	0.8559

	5								
Dummy (most frequent)	0.2247	0.0005	0.5	0.3333	0.1667	0.3333	0.2222	0.5	0
XGBoost regressor + cut points	--	--	0.8148	0.8232	0.7737	0.8232	0.7848	--	0.8983

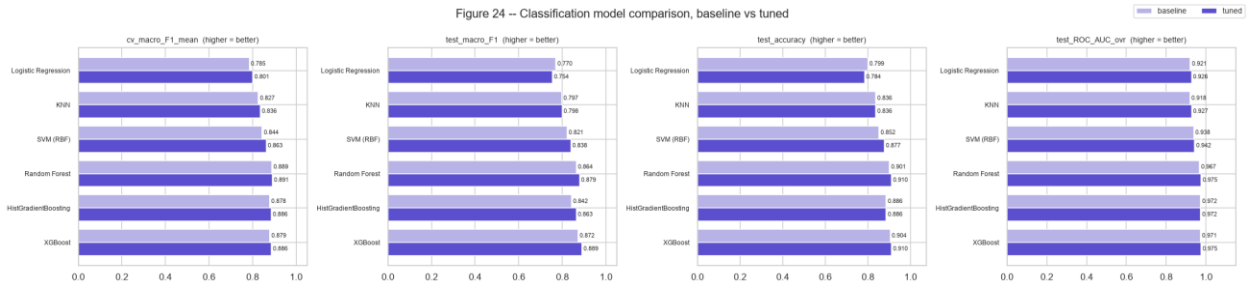


Figure R17 -- Classification metrics, baseline vs tuned.

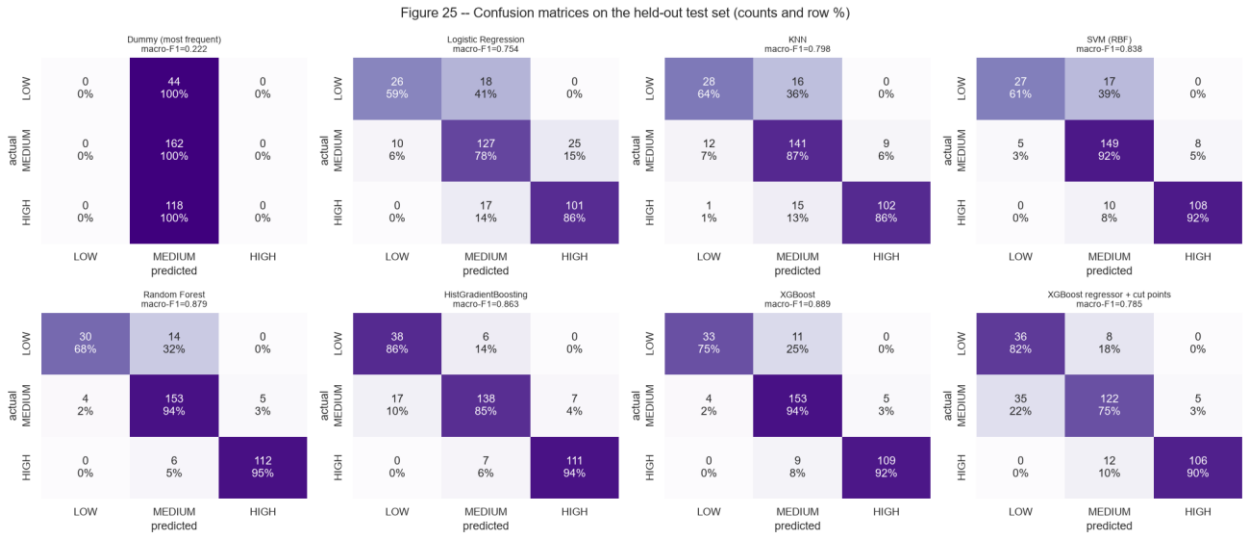


Figure R18 -- Test-set confusion matrices for every tuned classifier (counts and row %).

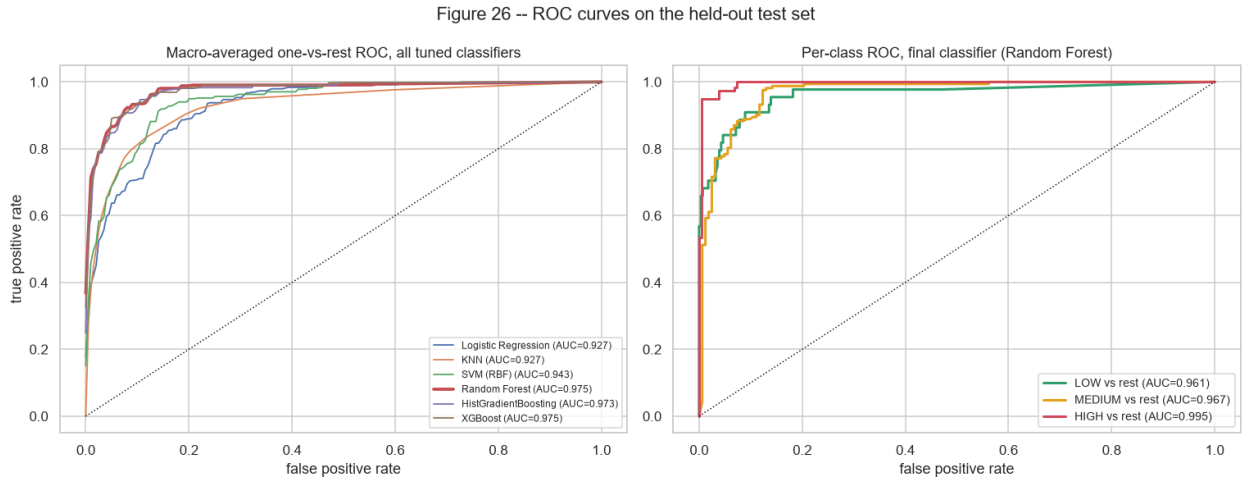


Figure R19 -- One-vs-rest ROC curves: all models (left) and per class for the final classifier (right).

Table 13 -- Test F1 per risk tier

model	LOW	MEDIUM	HIGH
Random Forest	0.7692	0.9134	0.9532

HistGradientBoosting	0.7677	0.8818	0.9407
XGBoost	0.8148	0.9134	0.9397
XGBoost regressor + cut points	0.6261	0.8026	0.9258
SVM (RBF)	0.7105	0.8817	0.9231
KNN	0.6588	0.8443	0.8908
Logistic Regression	0.65	0.784	0.8279

The row labelled '... regressor + cut points' classifies each booking by applying the tier cut points to the best regressor's predicted amount. It reaches only 0.785 macro-F1. About 29% of bookings sit exactly on a cut point (20 or 35 units), so a regression error of a single unit flips the tier. A dedicated classifier learns these boundaries directly, which justifies keeping two separate models.

8.3 Strengths and limitations of each model

Table 14 -- Strengths and limitations observed in this study

Model	Strengths	Limitations
Linear / Logistic Regression	Fast, transparent coefficients, stable across folds	Additive form cannot represent the pricing x service interaction or tier-dependent guest slopes; lowest scores among the real models
KNN	No training phase; exploits the booking grid	Distance is diluted by many one-hot columns; weakest on the minority LOW tier
SVR / SVM (RBF)	Largest gain from tuning; smooth surface	Slow to tune; probability outputs need an internal CV; still behind the ensembles
Random Forest	Strongest default model; best HIGH-tier F1; little tuning needed	Large artefact on disk; averaging blunts sharp steps in the target
HistGradientBoosting	Fast histogram boosting, competitive with no feature work	Most sensitive to the random-search draw; slightly behind XGBoost
XGBoost (hist)	Best regressor on CV and test; best LOW-tier and test macro-F1; explicit L1/L2/gamma regularisation	Largest hyperparameter space; under-performs with default settings

9. Final Model Selection

Table 15 -- Selected models and corrected resampled t-test against the runner-up (5x3 repeated CV)

task	selected	runner-up	metric	CV mean (selected)	CV mean (runner-up)	difference	t	p-value	folds won
regression	XGBoost	Random Forest	r2	0.9182	0.913	0.0052	1.1526	0.2684	11/15
classification	Random Forest	XGBoost	f1_macro	0.8865	0.8838	0.0027	0.4055	0.6912	8/15

9.1 Regression: XGBoost (hist)

The tuned XGBoost regressor has the highest cross-validated R2 (0.9181 +/- 0.0122). On the untouched test set it scores R2 = 0.9316, RMSE = 2.71 units, MAE = 1.89 units and MSE = 7.37, and 89.8% of test bookings are predicted within 5 units of the actual waste. Against the noise ceiling of R2 ~ 0.9328

estimated in Review 2, the model captures 99.9% of the explainable variance. It beats Random Forest on 11 of 15 repeated-CV folds. The corrected t-test (p = 0.27) shows the margin is not statistically decisive, but XGBoost also leads on every test metric and adds explicit regularisation and native per-prediction explanations (TreeSHAP contributions), which the dashboard uses.

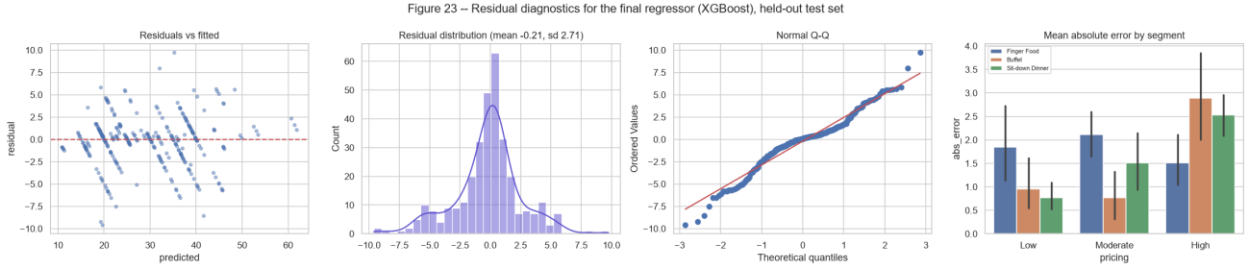


Figure R20 -- Residual diagnostics of the final regressor on the test set.

9.2 Classification: Random Forest

Table 16 -- Classification report of the final classifier (test set)

class	precision	recall	f1-score	support
LOW	0.8824	0.6818	0.7692	44
MEDIUM	0.8844	0.9444	0.9134	162
HIGH	0.9573	0.9492	0.9532	118
accuracy	0.9105	0.9105	0.9105	0.9105
macro avg	0.908	0.8585	0.8786	324
weighted avg	0.9107	0.9105	0.9083	324

Table 17 -- Confusion matrix of the final classifier (test set)

actual \ predicted	LOW	MEDIUM	HIGH
LOW	30	14	0
MEDIUM	4	153	5
HIGH	0	6	112

Under the pre-registered rule, the tuned Random Forest is selected with CV macro-F1 0.8908 +/- 0.0256. On the test set it reaches accuracy 0.910, macro precision 0.908, macro recall 0.859, macro-F1 0.879, ROC-AUC 0.975 and HIGH-tier recall 0.949. The runner-up, XGBoost, is statistically indistinguishable (corrected t-test p = 0.69). Both models are therefore valid choices, and the dashboard lets the user switch to any tuned model. Errors fall almost entirely between adjacent tiers: the classifier never confuses LOW with HIGH, so a wrong tier changes the recommended reduction by at most one step.

Figure 28 -- What drives the predictions

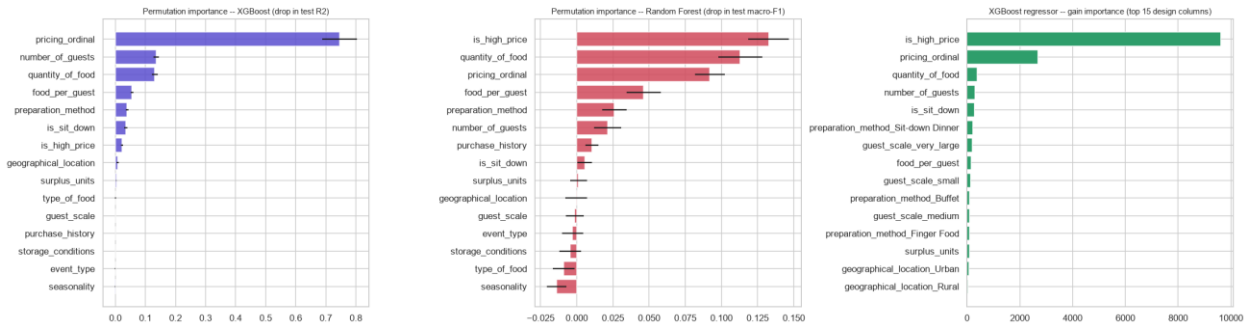


Figure R21 -- Grouped permutation importance of the final models (test set) and XGBoost gain importance.

Grouped permutation importance shuffles all one-hot columns of a business feature together. It confirms the EDA: pricing_ordinal (0.746), number_of_guests (0.136), quantity_of_food (0.131), food_per_guest (0.055), preparation_method (0.039), is_sit_down (0.035) account for almost all of the regressor's test R2. Food type, event type, storage, purchase history and seasonality contribute essentially nothing, as Section 3 predicted.

Figure 29 -- Learning curves of the final models

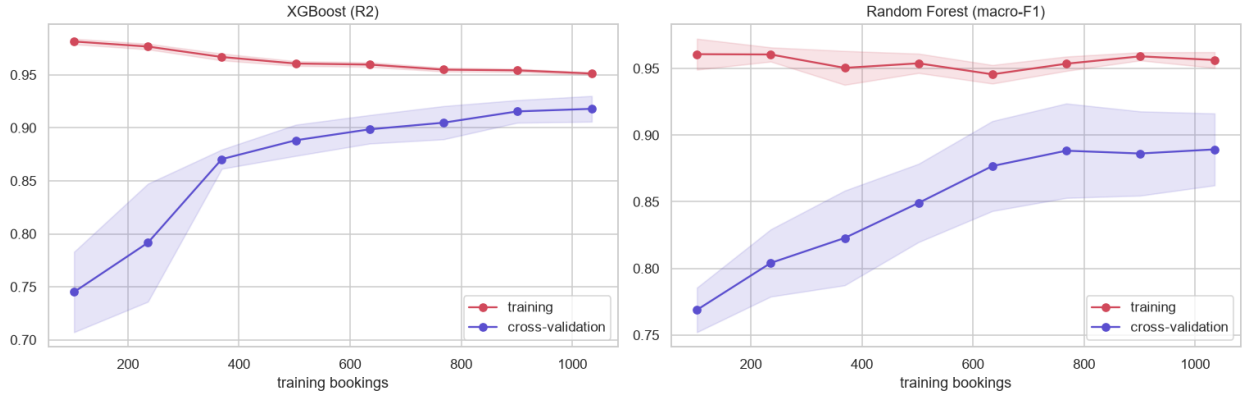


Figure R22 -- Learning curves of the final models.

The training and validation curves converge and the validation curve flattens. Adding more bookings of the same kind would bring little gain; the remaining error is mostly label noise, consistent with the ceiling analysis.

9.3 Why the selected solution is better

- **Accuracy:** the final regressor reduces test RMSE from 10.39 (naive) and 5.34 (linear) to 2.71 units.
- **Robustness:** the selection was made on cross-validation, and the test set confirms it rather than deciding it. Fold-to-fold spread is small (fold-score plots in Section 8).
- **Operational fit:** the classifier recalls 95% of HIGH-waste bookings, the ones that trigger a donation alert, with no LOW/HIGH confusions.
- **Interpretability:** the importance rankings match domain expectations and the EDA, and the dashboard explains every individual forecast.

10. End-to-End Product: Decision-Support Dashboard

The trained models are deployed in a Streamlit dashboard (app/streamlit_app.py). It runs the full chain: Booking Information -> Waste Prediction -> Risk Classification -> Preparation Recommendation -> Donation Alert. Inference goes through pipeline.prepare_inputs, the same feature engineering used in training, so the dashboard cannot drift from the models.

Table 18 -- Dashboard tabs

Tab	Function
Forecast	Booking form -> predicted waste, waste %, waste per guest, risk tier with class probabilities, recommended action, suggested reduced quantity with the re-forecast waste, donation-partner alert, segment benchmark, a what-if curve of waste vs quantity prepared, a per-feature explanation (XGBoost TreeSHAP contributions) and every tuned model's forecast for the same booking
Data Explorer	Interactive EDA over every column: filters, KPIs, waste / waste rate / HIGH share by any column with 95% CI, risk-tier mix, density curves, a two-column heatmap, a scatter explorer, statistical tables and the full EDA figure gallery
Model Performance	Final model KPIs, tuned and baseline tables, before/after tuning, significance test, per-tier F1 heat map, best hyperparameters and the model figure gallery
Batch Forecast	Upload a CSV of bookings (original or snake_case column names), get forecasts, actions, suggested quantities and alerts, and download the results
About	Pipeline, methodology and limitations

The sidebar defaults to the selected models (XGBoost regressor, Random Forest classifier) and allows switching to any tuned model. A warning appears when a booking falls outside the training range (guests 207-491, quantity 280-500), because tree models do not extrapolate.

Launch: ./venv/Scripts/python -m streamlit run app/streamlit_app.py

10.1 Prescriptive action engine

Table 19 -- Tier-to-action mapping

Predicted tier	Action	Dashboard behaviour
LOW	Normal preparation	--
MEDIUM	Reduce quantity of food by 10-15%	Suggested quantity uses the 12.5% midpoint
HIGH	Reduce quantity by 15-20% and flag a donation-partner alert	Suggested quantity uses the 17.5% midpoint; alert raised

11. Conclusions

- Waste can be forecast before an event with test R2 = 0.932 (RMSE 2.71 units) by a tuned XGBoost (hist) regressor. This is close to the ceiling set by label noise.
- Booking risk tiers are classified with macro-F1 = 0.879 and ROC-AUC = 0.975. Errors are confined to adjacent tiers.
- Tree ensembles clearly outperform linear, instance-based and kernel models, because waste depends on a pricing x service interaction and on tier-specific guest effects that additive models cannot represent.
- Hyperparameter tuning matters most for XGBoost and SVR. Random Forest and HistGradientBoosting are near-optimal at their defaults.
- Of the ten booking attributes, pricing tier, preparation method, guest count and quantity prepared carry almost all of the predictive signal. Event type, food type, storage, season and purchase history add little or nothing.
- The project is delivered as a working dashboard that turns each forecast into a concrete preparation quantity and a donation alert.

12. Limitations and Future Work

- The dataset has limited value granularity: 61 guest/quantity combinations, and most waste values are multiples of 5. Performance should be re-validated on a caterer's own operational records before deployment.
- No date field exists, so weekday, holiday and lead-time effects cannot be modelled. Adding the booking date is the most valuable data extension.
- Pricing is a three-level tier, which limits price-sensitivity analysis to tier comparisons.
- The projected waste at a reduced quantity is an in-sample what-if, not a causal estimate. A controlled pilot is needed to measure the real effect of the recommended reductions.
- Forecasts support staff decisions and never override contract minimums, dietary requirements, service buffers or food-safety rules.
- Future work: per-tier prediction intervals (quantile XGBoost), cost-sensitive thresholds that weight a missed HIGH booking more heavily, and periodic retraining as new bookings arrive.

13. Project Artefacts

Table 20 -- Deliverables

Path	Content
src/pipeline.py	Review 2 pipeline: quality audit, cleaning, split, tiers, features, integration, EDA
src/eda_extended.py	Extended EDA across every column (Figures 8, 11-20; eda_*.csv)
src/train_models.py	Baseline + tuned training, evaluation, comparison, significance, figures 21-31, model export
src/build_notebooks.py, src/review3_cells.py	Builds and executes the Review 2 and Review 3 notebooks
src/build_report.py	Generates this report from the result files
notebooks/Review3_Model_Building_and_Comparison.ipynb	Executed Review 3 notebook
out/*.csv, out/best_hyperparameters.json	All result tables
models/	regressor.pkl, classifier.pkl, model_zoo.pkl (all tuned models), metadata.json
app/streamlit_app.py	Interactive dashboard

References

1. Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proc. KDD.
2. Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS (histogram-based boosting).
3. Breiman, L. (2001). Random Forests. Machine Learning 45(1).
4. Nadeau, C. and Bengio, Y. (2003). Inference for the Generalization Error. Machine Learning 52(3).
5. Bergstra, J. and Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. JMLR 13.
6. Bergsma, W. (2013). A bias-correction for Cramer's V and Tschuprow's T. J. Korean Statistical Society 42(3).
7. Lundberg, S. et al. (2020). From local explanations to global understanding with explainable AI for trees. Nature MI 2.
8. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
9. Kaggle dataset: Food Wastage Data in Restaurant (trevinhannibal).